

EDTS APM 2027

Software Requirements Specification

Franchise Application and Review Platform

Draft 0.2 | 1 October 2026 | Prepared by Adi Haditya Nursyam

Contents

1. Introduction.....	2
1.1 Purpose and baseline.....	2
1.2 Problem statement and pain-point analysis.....	2
1.2.1 Core problem.....	2
1.2.2 How gaps were identified.....	2
1.2.3 Six headline problems	3
1.2.4 Applicant pain points.....	3
1.2.5 Franchisor pain points	5
1.2.6 Problems affecting both sides.....	8
1.2.7 How findings shaped and built the prototype	9
1.2.8 Scope boundary	10
1.3 Terms.....	10
2. Application	11
2.1 Users and main flows.....	11
2.2 Architecture and external services	11
2.3 Data and storage boundaries	12
3. Product Specification	12
3.1 Functional requirements.....	12
3.2 Quality and operational requirements.....	13
3.3 Key source paths	14
4. Assumptions and Decisions.....	14
4.1 Assumptions.....	14
4.2 Decisions before production.....	14
5. References	15
6. Attachments (AI Sessions).....	15
6.1 Audit_Dev Session.....	16
6.2 Build_Process Session.....	22
6.3 Doc_Spec Session.....	34

1. Introduction

1.1 Purpose and baseline

This specification records the intended users, scope, system behavior, quality constraints, and open decisions for EDTS APM 2027. It is an initial baseline for a franchise application and review prototype. Its structure follows the IEEE Computer Society SRS guidance on clear, testable, prioritized requirements and on separating functional requirements from quality constraints [1]. It does not claim formal standards conformance.

Status describes checked repository source, not a live deployment. Source means behavior found in code. Partial means an incomplete workflow. Local only means data remains in this browser. Config means an external service or credentials are required. Gap means no matching capability was found. Evidence labels distinguish visible observations from product inferences.

1.2 Problem statement and pain-point analysis

1.2.1 Core problem

The public form collects applicant information, but the visible experience is centered on one submission. It gives applicants little visible support for preparing information, saving progress, understanding status, or completing follow-up actions. The supplied notes also show that much of the material reviewers need arrives as free text. The resulting product problem is that the form collects data but does little to turn it into a clear application and review process.

This statement describes the public form and the product opportunity. It does not assert that McDonald's internal tools lack status tracking, anti-spam controls, or reviewer workflows.

1.2.2 How gaps were identified

The analysis combines the product owner's observations of the public form with a source review of this prototype. It did not include access to McDonald's internal systems or interviews with its applicants or reviewers.

Pass	Method and result
1. Observe the public form	Record the visible labels, fields, input types, currency, resume instruction, and visible actions. Treat the supplied form notes as the observation record.
2. Map both journeys	Follow applicant steps from program discovery through post-submission actions. Map reviewer needs from intake through triage, revision, proposal, and closure.
3. Separate evidence from inference	Mark page-level observations as visible gaps. Mark reviewer workload, scale effects, and missing internal tools as inferences or unverified risks.
4. Trace the prototype	Inspect route handlers, UI components, domain logic, database schema, and browser storage. Confirm the real behavior and its persistence boundary.
5. Convert gaps into requirements	Map each pain point to a product response, functional requirement, and code status. Keep unbuilt opportunities visible as gaps rather than calling them implemented.

Visible means supported by the public page observations. Inferred means a likely workflow or scale effect that must be validated with intended users. Unverified risk means the available page does not show whether a hidden control or internal tool exists. Prototype status describes source code and storage behavior, not runtime or deployment acceptance.

1.2.3 Six headline problems

1. **High application friction.** Applicants manually re-enter information already present in a resume.
2. **Poor document experience.** The visible form asks applicants to paste a resume instead of uploading the original document.
3. **No application visibility.** The visible flow treats submission as an event and provides no workspace for status or next actions.
4. **Unstructured applications create reviewer workload.** Education, business experience, comments, and resumes rely heavily on free text.
5. **No efficient application triage.** Reviewers need search, screening, prioritization, and summaries before detailed review.
6. **Application operations are fragmented.** Screening, status, revisions, communication, proposals, and approval need a shared process.

1.2.4 Applicant pain points

The evidence label appears in the first column. Each response names the prototype behavior and its current limit. Requirement IDs point to Section 3.1.

Pain point and evidence	Gap or user impact	Prototype response and status
AP-01 [Visible] Too much manual data entry	Visible form asks applicants to type profile, education, and business experience. Re-entry adds effort and can introduce errors.	Resume extraction can prefill supported answers for applicant review. PDF, JPG, and PNG only, up to 5 MB. Status: Config, partial. Related: FR-06.
AP-02 [Visible] Resume handling is poor	The visible form asks applicants to paste a resume into a text field, which loses the original document structure.	The prototype accepts supporting files and can extract from a resume. The original file stays in browser IndexedDB. Status: Config, Local only. Related: FR-06, FR-07.
AP-03 [Visible] Repeated information	Education and business experience may already appear in a resume, so applicants repeat information.	Resume extraction can reuse supported fields, but there is no complete cross-application profile reuse. Status: Config, partial. Related: FR-05, FR-06.
AP-04 [Visible] No visible applicant account or workspace	The public form shows no applicant workspace for managing a submission.	Google sign-in, an application list, and application details exist in the configured path. Status: Source, Config. Related: FR-02.

Pain point and evidence	Gap or user impact	Prototype response and status
AP-05 [Visible] No save and resume workflow	A long submission may require information the applicant does not have in one sitting.	The form saves answers and the current step in browser localStorage. It does not sync drafts across devices. Status: Local only. Related: FR-05.
AP-06 [Visible] No visible progress indicator	The observed form is a single page and does not show progress through application steps.	The prototype uses a multi-step form with progress. Status: Source. Related: FR-04.
AP-07 [Visible] No post-submission status tracking	The visible flow does not show a workspace for checking whether a submission is under review or needs action.	Applicants can see a lifecycle stage and revision or proposal prompts. There is no event timeline. Status: Partial. Related: FR-08, FR-10.
AP-08 [Visible] No action centre	The observed form has no applicant area that collects pending actions after submission.	Revision and proposal actions appear on each application detail. There is no consolidated action centre. Status: Partial. Related: FR-11, FR-13.
AP-09 [Visible] No structured revision flow	The public flow shows no way to reopen specific answers after submission; corrections may require separate contact.	Reviewers can request named fields or documents with notes, and applicants can return the revised application. Status: Source. Related: FR-11.
AP-10 [Visible] Poor document support	The observed form provides no visible upload checklist for a resume, financial evidence, or site photos.	Program forms accept multiple files and show document status. Files remain in the current browser. Status: Local only. Related: FR-07.
AP-11 [Visible] Limited context around questions	Some visible fields provide little guidance about what a complete answer should include.	Program fields support help text, options, and conditional display. Example guidance is not yet complete. Status: Partial. Related: FR-03, FR-04.
AP-12 [Visible] Weak financial input experience	The form labels investment in USD at section level and presents a single cash-investment field.	Example programs use IDR choices. Final ranges, currency formatting, and eligibility wording still need approval. Status: Partial. Related: FR-17.
AP-13 [Visible] Global rather than localized experience	The visible form asks about country and uses USD, which leaves Indonesian applicants to interpret a global form.	The prototype uses Indonesian labels, IDR options, and local example fields. Business rules need owner approval. Status: Source, pending approval. Related: FR-04, FR-17.

Pain point and evidence	Gap or user impact	Prototype response and status
AP-14 [Visible] Little clarity about what happens next	The visible journey ends at submission without a stated review timeline.	The workspace shows application stage and per-application actions. It does not promise a service timeline. Status: Partial. Related: FR-08, FR-10, FR-11.
AP-15 [Visible] No communication history	Follow-up conversations can become detached from the original submission.	An application message panel exists, but messages stay in this browser and no email is sent. Status: Local only. Related: FR-15.
AP-16 [Visible] No application history	The visible form has no central list for returning applicants to review current and past applications.	The applicant workspace lists applications for the signed-in account in the configured path. Status: Source, Config. Related: FR-02, FR-08.
AP-17 [Visible] No program discovery or comparison flow	The observed path begins at the form instead of helping an applicant browse opportunities first.	The prototype has an open-program directory. It does not provide side-by-side program comparison. Status: Partial. Related: FR-01.
AP-18 [Visible] Generic form experience	The public form behaves like a basic web form rather than a guided application process.	A branded Indonesian portal presents program-specific multi-step forms. Status: Source. Related: FR-04.

1.2.5 Franchisor pain points

These operational effects are inferences from the visible form and likely review tasks. They are not claims about McDonald's internal tools. Duplicate and spam controls are explicitly unverified.

Pain point and evidence	Gap or reviewer impact	Prototype response and status
FP-01 [Inferred] No persistent applicant identity in the visible workflow	The public form exposes no authenticated identity layer. Continuity across submissions cannot be confirmed from the form.	Google OAuth assigns applicant or franchisor roles in the configured path. Duplicate linking is not implemented. Status: Partial, Config.
FP-02 [Unverified risk] Duplicate or noisy submission risk	A public entry point can create an operational risk at volume. The supplied review did not inspect hidden spam controls.	Account sign-in adds identity, but duplicate detection and rate controls are absent from this repository. This risk needs validation. Status: Gap.
FP-03 [Inferred] Unstructured resume data	A pasted resume is a large text block that reviewers must interpret consistently.	Gemini can extract supported fields and prepare a review brief. Original files remain browser-local. Status: Config, Partial.

Pain point and evidence	Gap or reviewer impact	Prototype response and status
FP-04 [Inferred] Education is difficult to filter	Free-text education summaries do not consistently separate degree, school, or field of study.	The example form includes an education-level choice and resume extraction. School and degree normalization is absent. Status: Partial.
FP-05 [Inferred] Business experience is difficult to compare	Long experience descriptions require manual interpretation across applicants.	The form captures business type and duration plus a narrative answer. AI summaries are optional. A standard rubric is absent. Status: Partial, Config.
FP-06 [Inferred] Limited search and filtering	Questions about investment capacity or experience are hard to answer from raw text.	The list searches name, reference, email, city, and program, and filters by stage. It does not filter by investment or experience. Status: Partial.
FP-07 [Inferred] Every application initially demands human attention	A reviewer may need to open each incomplete or unsuitable record before deciding what to do.	Required-field checks and configured screening rules support initial triage. Rule quality depends on owner approval. Status: Source.
FP-08 [Inferred] No fast prioritization	A queue does not automatically show which applications need attention first.	Rule outcomes feed a deterministic score. The score is not validated as an eligibility measure, and a hard fail can reject automatically. Status: Partial.
FP-09 [Inferred] Repetitive screening work	Reviewers repeatedly read similar profile, finance, and experience information.	An optional Gemini review can summarize answers and supported documents. Manual review remains available. Status: Config.
FP-10 [Inferred] Missing or contradictory data is hard to detect	Reviewers may have to find omissions or inconsistencies themselves.	Required fields and configured checks flag some missing values. Cross-field contradiction detection is not implemented. Status: Partial.
FP-11 [Inferred] No unified review brief	Relevant facts are spread across fields and long responses.	Application detail presents screening findings and a summary; Gemini can add a review brief. It is not a verified or normalized case file. Status: Partial, Config.
FP-12 [Inferred] No structured pipeline	A response form alone does not show application movement across review stages.	The franchisor has a dashboard, searchable table, and stage board with selector or drag and drop. Status: Source.

Pain point and evidence	Gap or reviewer impact	Prototype response and status
FP-13 [Inferred] Spreadsheet-style management scales poorly	Sorting records alone does not assign owners or coordinate work across a team.	The prototype provides a searchable table and board. Bulk actions, export, and reviewer assignment are absent. Status: Partial.
FP-14 [Inferred] No visible workflow state model	Stages such as screening, interview, and proposal need a shared definition.	The code defines eight common stages. Their names and order are not configurable per program. Status: Partial.
FP-15 [Inferred] No clear queue of required work	A reviewer may not know which records are waiting for their action.	The overview shows recent records and stage counts, but has no assigned task queue or service-level timer. Status: Partial.
FP-16 [Inferred] Follow-ups are easy to forget	Staff must remember which applicants have had no recent activity.	A reminder becomes due after five days and can be recorded locally as sent. No reminder is delivered. Status: Local only.
FP-17 [Inferred] Email is detached from application context	Reviewers may switch tools and manually recreate application context for an email.	There is no email integration in the checked source. Status: Gap.
FP-18 [Inferred] Repeated email writing	Common revision or proposal messages may need to be written repeatedly.	There are no email templates or email delivery service. Status: Gap.
FP-19 [Inferred] No shared communication record	A team cannot rely on a browser-local conversation as a common record.	Messages are stored in localStorage and are not synchronized to the database workspace. Status: Local only.
FP-20 [Inferred] No revision workflow	Requesting one correction by email can create a thread that is hard to reconcile.	Reviewers can request specific fields or documents with notes. Status: Source.
FP-21 [Inferred] Poor document review experience	Pasted text does not preserve document layout, and local files are not shared with other devices.	Reviewers can open locally stored files and optional AI findings. Private shared file storage is absent. Status: Local only, Partial.
FP-22 [Inferred] No structured proposal flow	A review may move to separate documents or email when terms are discussed.	A proposal screen captures terms and applicant responses. Attachments retain names, not file content. Status: Partial.
FP-23 [Inferred] No applicant comparison view	Individual reviews make relative comparison harder.	The repository has a list and pipeline, but no side-by-side candidate comparison. Status: Gap.

Pain point and evidence	Gap or reviewer impact	Prototype response and status
FP-24 [Inferred] Limited operational analytics	The public form itself offers no view of volume or stage distribution.	The overview shows active, new, revision, average-score, and stage counts. It is not a full analytics report. Status: Partial.
FP-25 [Inferred] Hard to change application requirements	A fixed form may require code changes when programs ask different questions.	The program editor configures fields, options, visibility conditions, and screening rules per program. Status: Source.
FP-26 [Inferred] Hard to change the review process	Different programs may need stages that do not match a single shared pipeline.	Stages are currently fixed in code and shared by all programs. Status: Gap.

1.2.6 Problems affecting both sides

These shared problems describe the proposed product opportunity, not verified defects in an internal McDonald's workflow.

Pain point and evidence	Shared consequence	Prototype response and status
SP-01 [Inferred] No shared source of truth	Applicants may not see current status while reviewers track work elsewhere. This is a workflow inference, not a claim about McDonald's internal system.	Configured accounts share program and application records through the database. Drafts, files, and messages remain local. Status: Partial.
SP-02 [Inferred] No application lifecycle	A submit-only flow does not explain review, revision, proposal, or closure.	The prototype defines eight stages and applicant status views. Email and configurable stage policy are absent. Status: Partial.
SP-03 [Inferred] No structured communication	Applicants and reviewers may rely on separate channels with no shared context.	A per-application message panel exists but is browser-local and sends no email. Status: Local only.
SP-04 [Inferred] No revision loop	Applicants may resend information while reviewers reconcile versions.	Named revision requests and resubmission create a basic in-app loop. Status: Source.
SP-05 [Inferred] No shared document workspace	Applicants need a reliable upload location and reviewers need access to the same document.	Files are held in browser IndexedDB, not authenticated shared storage. Status: Local only, Partial.

Pain point and evidence	Shared consequence	Prototype response and status
SP-06 [Inferred] No clear status model	Applicants face uncertainty while reviewers use inconsistent labels.	The application has defined stages shown to both sides. Status: Source.
SP-07 [Inferred] No user identity or account	Applicants cannot visibly return to a personal workspace from the public form.	Google OAuth and applicant or franchisor roles exist in the configured path; local demo access is development-only. Status: Config, Source.
SP-08 [Inferred] No structured screening	Repeated checks increase manual effort and make outcomes harder to explain.	Enabled program rules return pass, warning, or reject results with reasons. Rule content and rejection policy need approval. Status: Partial.
SP-09 [Inferred] No reusable applicant profile	Applicants may repeat information already supplied in earlier materials or submissions.	Resume extraction can prefill supported answers. A complete cross-device profile is not implemented. Status: Config, Partial.
SP-10 [Inferred] Static form for different programs	One fixed question set can fail to capture program-specific needs.	Franchisors can configure fields and rules by program. Stages remain shared and fixed. Status: Partial.

1.2.7 How findings shaped and built the prototype

The product response turns the single submission into a sequence: discover a program, complete a guided application, submit and track it, resolve a targeted revision, and respond to a proposal. The reviewer side adds configurable program questions, basic screening, a queue and pipeline, review details, revisions, and proposals. The table traces those choices to source files and keeps incomplete work visible.

Gap area	Built response	Source and remaining limit
Program discovery and localization	A directory presents open example programs before the form. Program labels and sample investment choices use Indonesian and IDR.	app/page.tsx; components/applicant/program-directory.tsx; lib/demo-data.ts. The examples and eligibility rules need business approval. FR-01, FR-04, FR-17.
Guided application and resume support	The form renders program fields as steps, validates answers, saves a local draft, stores files in IndexedDB, and can ask Gemini to prefill supported fields from a resume.	components/applicant/application-form.tsx; lib/demo-files.ts; app/api/application-review/route.ts. Drafts and files are local; extraction is configured and lacks a separate consent step. FR-04 to FR-07.

Gap area	Built response	Source and remaining limit
Identity and workspace	Better Auth defines applicant and franchisor roles. In the configured non-development path, the workspace API loads and saves program and application records with ownership checks.	lib/auth.ts; lib/db-schema.ts; app/api/workspace/route.ts; components/workspace-sync.tsx. OAuth, database, and deployment must be configured and verified. FR-02, FR-16.
Screening and review	Deterministic rules check required fields and configured conditions. An optional Gemini route can prepare a review summary, document findings, and questions.	lib/screening.ts; app/api/application-review/route.ts; components/franchisor/application-review.tsx. Scores and rules need owner approval; AI is not market verification. FR-09, FR-12.
Reviewer operations	A dashboard, searchable table, stage filters, and pipeline show workload and move records through fixed stages.	components/franchisor/overview.tsx; components/franchisor/application-list.tsx; components/franchisor/pipeline.tsx. Stages are not configurable and comparison or assignment queues are absent. FR-10.
Revisions and proposals	Reviewers can request named changes. Applicants can respond to revisions and proposals; approval requires both parties to accept.	components/applicant/application-detail.tsx; components/franchisor/proposal-editor.tsx. Proposal attachments retain names only. FR-11, FR-13, FR-14.
Communication and follow-up	The detail page has a message panel and a reminder control that becomes due after five days.	components/communication-panel.tsx; lib/demo-communication.ts. Messages and reminders stay in localStorage. The control records a reminder and sends no email. FR-15, FR-16.

The prototype therefore addresses the visible form friction and establishes a basic workflow. It does not yet provide shared file storage, delivered email, configurable stages, duplicate detection, or a complete reviewer task queue. These remain gaps for validation and future scope decisions.

1.2.8 Scope boundary

This is an Indonesian concept prototype informed by the public McDonald's franchise form. It is not an official or affiliated McDonald's service. Payments, contracts, final franchise decisions, verified market research, internal-system integration, spam-control verification, and production operation are outside the current baseline.

1.3 Terms

Term	Meaning
Applicant	A person who creates and manages a franchise application.

Term	Meaning
Franchisor	The organization and authorized reviewers who manage programs and applications.
Program	A franchise opportunity with its own application fields and screening rules.
Application	An applicant's answers, documents, review state, and any proposal for one program.
Screening	An initial rules-based check. A score is not proof of eligibility.
SRS	Software Requirements Specification.
Visible gap	A missing or inconvenient feature on the public form reviewed. It does not prove that internal tools lack the feature.
Inferred pain	A likely workflow problem deduced from the visible form and requiring validation with intended users.
Unverified risk	A concern, such as duplicate or spam submissions, that cannot be confirmed from the public page.

2. Application

2.1 Users and main flows

Public visitor. Browse open program descriptions.

Applicant. Sign in, choose a program, complete or save an application, attach requested information, track status, respond to a revision, review a proposal, and cancel before a proposal or approval.

Franchisor reviewer. Configure programs and screening rules, search the application queue, review rule or AI findings, move cases between stages, request revisions, and prepare proposals. The source defines applicant and franchisor roles, with no separate system-administrator role.

Applicant flow: program directory, sign-in, guided application, submission, status tracking, revision or proposal response. Reviewer flow: dashboard, searchable queue or pipeline, review, revision request or proposal, and status update.

2.2 Architecture and external services

The web application uses Next.js, React, TypeScript, Better Auth, Drizzle ORM, and libSQL/Turso. In the configured production path, Google OAuth provides sign-in and the workspace API stores program and application records with role and ownership checks. Development mode also contains seeded demo identities and browser-local records.

Google Gemini is called by a server route for user-initiated resume extraction and optional application review. Missing credentials or failed analysis return to manual entry or rule-based review. Site coordinates can open in Google Maps; no traffic, demographic, competitor, or site-eligibility data is verified. No email, payment, or McDonald's API integration is present in the checked source.

2.3 Data and storage boundaries

The database schema covers users, programs, and applications. Application documents are stored in browser IndexedDB. Application drafts, proposal drafts, messages, and reminder records use browser localStorage. Proposal attachments currently retain file names rather than file content. These local records are not shared across devices or users.

The current program form collects personal, experience, funding, location, legal-identity, and consent information. Production use requires approved purposes, access rules, retention and deletion periods, and clear disclosures for identity data and third-party AI processing.

3. Product Specification

3.1 Functional requirements

Each statement is a proposed target. The repository-status column shows the current source baseline, not verified deployment readiness.

ID / priority	Requirement	Status
FR-01 MUST	Applicants shall see open programs with their summary, operating model, target locations, and a route to the matching application.	Source
FR-02 MUST	Production users shall sign in with Google and receive an applicant or franchisor role. Protected pages and records shall follow that role and ownership.	Source
FR-03 MUST	Franchisors shall create, edit, open, close, and remove programs; configure fields, sections, options, basic visibility conditions, and screening rules.	Source
FR-04 MUST	The form shall render Indonesian multi-step sections from the program schema, show progress, validate required and typed values, and label investment options in IDR.	Source
FR-05 SHOULD	Applicants shall save answers and the current step, then resume the draft in the same browser. Drafts and selected files currently do not sync across devices.	Local only
FR-06 SHOULD	On user request, the system shall read a PDF, JPG, or PNG resume up to 5 MB, prefill supported fields, and let the applicant verify or correct every value. Manual entry shall remain available.	Config
FR-07 MUST	Applicants shall attach required or optional program documents. The current client allows PDF, DOC, DOCX, XLS, XLSX, JPG, JPEG, PNG, and ZIP files up to 10 MB each. The target is authorized cross-device file access.	Local only
FR-08 MUST	A submission shall require an open program and valid visible fields, then record the answers, program, reference, time, screening result, and lifecycle stage in the applicant workspace.	Source

ID / priority	Requirement	Status
FR-09 MUST	The system shall evaluate required fields and enabled present, equals, contains, and gte rules as pass, warning, or reject, with an explainable triage score. Current reject rules can set the stage to Rejected automatically.	Source
FR-10 MUST	Franchisors shall review a dashboard, search or filter applications, inspect each record, and move records through the stage pipeline by selector or drag and drop.	Source
FR-11 MUST	A reviewer shall request named fields or documents with notes. The applicant shall update the requested items and return the application to review.	Source
FR-12 SHOULD	With the separate AI review permission enabled, the server may send selected answers and eligible business or financial files to Gemini for a summary, strengths, concerns, questions, and supported document or location findings. Rules and manual review shall remain available.	Config
FR-13 SHOULD	Franchisors shall prepare structured proposals. Applicants shall accept, reject, or request changes. The application shall be approved only after both parties accept.	Partial
FR-14 SHOULD	Applicants shall be able to cancel an application before a proposal or approval. Withdrawal shall remain distinct from a franchisor rejection.	Partial
FR-15 SHOULD	Application messages and follow-up reminders shall persist across authorized sessions and accurately report delivery. Current messages and reminder records are local to the browser; no email is sent.	Local only
FR-16 MUST	Production shall persist account, program, and application records in the configured database and enforce applicant ownership and franchisor program ownership. Files, drafts, proposal attachments, and messages shall have defined shared storage.	Partial
FR-17 MUST	The interface shall use Indonesian labels, Rupiah investment options, and user-supplied site information. A map point or AI summary shall not be described as verified market research.	Source

Rule score in the current code is 100 minus 40 for each reject check and 12 for each warning, bounded from 0 to 100. A reject result currently sets the application stage to Rejected at submission. Confirm this policy before treating the score as a final eligibility decision.

3.2 Quality and operational requirements

NFR-01 Access control. The server shall authenticate production sessions and authorize applicant and franchisor access by role and record ownership. Development demo identities shall not be treated as production authentication. Source code includes these controls; deployment was not tested.

NFR-02 Personal data and AI. AI review has a separate permission field and excludes selected identity fields from its review payload. Resume extraction sends the original resume to Gemini when the applicant starts extraction; the current source has no separate extraction-consent step. Production requires a clear notice, explicit consent, and a provider-retention decision.

NFR-03 Human review. AI summaries and scores shall support review only. Source prompts instruct Gemini not to make final decisions. Deterministic hard-fail rules currently move a submission to Rejected, so the owner must confirm whether a reviewer must approve that outcome.

NFR-04 Localization. The current interface is Indonesian and uses IDR program labels. Business owners must approve the actual eligibility questions, monetary tiers, and legal wording before production.

NFR-05 Service levels. Latency, throughput, availability, backup, recovery, audit-log, accessibility, and data-retention targets are not specified in the source. Set measurable targets before production acceptance.

3.3 Key source paths

Authentication and record access: lib/auth.ts and app/api/workspace/route.ts. Applicant form and resume flow: components/applicant/application-form.tsx and app/api/application-review/route.ts. Screening: lib/screening.ts. Local files and communication: lib/demo-files.ts and lib/demo-communication.ts. Reviewer pipeline and proposals: components/franchisor/pipeline.tsx and components/franchisor/proposal-editor.tsx. Program configuration: components/program-admin/program-editor.tsx [3].

4. Assumptions and Decisions

4.1 Assumptions

A-01. This is an Indonesian concept prototype informed by the public McDonald's franchise form. It is not an official McDonald's system or a statement of McDonald's internal process.

A-02. The supplied form notes describe visible page behavior only. They do not establish whether hidden spam controls, internal application tracking, or reviewer tools exist.

A-03. The two programs, field schema, financial ranges, and screening rules in the repository are examples until the business owner approves them.

A-04. AI and rules provide support, not independent business, legal, financial, or site-selection decisions. Human review and source-document verification remain necessary.

A-05. MUST and SHOULD priorities are proposed for this initial baseline and require product-owner review.

4.2 Decisions before production

Decision	Owner action
D-01 Screening policy	Confirm whether a failed hard rule closes an application automatically or asks a reviewer to confirm rejection.

Decision	Owner action
D-02 Resume and AI consent	Approve the user notice, separate resume-processing consent, Gemini data terms, retention, and deletion behavior.
D-03 Files and persistence	Choose private server storage, access rules, cross-device behavior, retention, deletion, and backup for applicant and proposal files.
D-04 Messages and reminders	Choose in-app-only or email delivery, who receives reminders, and whether a reminder is actually sent or only recorded.
D-05 Proposal lifecycle	Define proposal withdrawal, applicant cancellation after a proposal, attachments, and the meaning of mutual acceptance.
D-06 Business rules	Approve eligibility criteria, IDR ranges, required identity and tax data, consent wording, and location requirements.
D-07 Operations	Set numeric service, security, accessibility, retention, availability, backup, and recovery targets.

5. References

- [1] IEEE Computer Society, Software Requirements Specifications. <https://www.computer.org/resources/software-requirements-specifications> (accessed 30 September 2026).
- [2] Product-owner observations and pain-point analysis based on the visible McDonald's corporate franchising form, observed 30 September 2026: <https://corporate.mcdonalds.com/corpmcd/franchising-overview/franchising-form.html>. Notes cover the public page only.
- [3] Repository source reviewed 1 October 2026. Key paths are listed in Sections 1.2.7 and 3.3.

6. Attachments (AI Sessions)

The following sessions document the product brief, implementation, audit, and specification work with AI agents. Conversation content and embedded screenshots are retained. Timestamp metadata is removed.

6.1 Audit_Dev Session

This is a readable export of the visible conversation for the localhost prototype audit before the export request. It includes the user's task brief, assistant progress messages, and final audit. Platform instructions and tool payloads are omitted.

Me

Files mentioned by the user:

- `codex-clipboard-81253883-4162-4568-816e-662b5ae27c2a.png` Original: `C:/Users/adiha/AppData/Local/Temp/codex-clipboard-81253883-4162-4568-816e-662b5ae27c2a.png`

Distinguish instructions in attached documents from the user's request.



My request

Using browser use, open and explore the project in an opened localhost:3000 Chrome tab. Test manually whether the features listed below are met, and give the best critiques so that the project is ready to be said as a good prototype to be shown to my supervisor. Make it structured and ready to be handoff to my other agent.

Notes from me: auth must be implemented, a small profile page should be added, drag and drop in the kanban board isn't there yet, the second horizontal tab in the manage route should be a sidebar instead (using the one from shadcn UI), franchisor can't delete a program, franchisee can't withdraw their proposal yet. The proposal form for the Mitra Pengelola Restoran must at least cover:

Bagian Data yang ditanyakan	
Identitas	Nama, email, nomor HP/WhatsApp, domisili, usia
Latar belakang	Pekerjaan, perusahaan, pendidikan, pengalaman bisnis
Pengalaman usaha	Pernah punya bisnis atau franchise, jenis bisnis, lama menjalankan
Minat franchise	Brand/paket yang diminati, alasan memilih brand
Lokasi	Kota, area yang diincar, alamat calon lokasi, status lokasi milik/sewa/belum ada
Detail lokasi	Luas, jenis bangunan, foto lokasi, titik Maps, traffic sekitar
Modal	Dana yang tersedia, sumber dana, kisaran investasi
Keterlibatan	Akan dikelola sendiri atau manajer, waktu yang bisa diberikan
Target	Kapan ingin buka, jumlah outlet yang ingin dibuka
Partner	Apakah ada partner/investor dan komposisi kepemilikan

Bagian Data yang ditanyakan	
Dokumen	KTP, NPWP/NIB jika ada, foto lokasi, bukti kepemilikan/sewa bila perlu
Pernyataan	Persetujuan penggunaan data dan konfirmasi bahwa data benar

I never tested the AI capability and usage, so perform test for it too. You may also see the codebase to make the best audit by combining it from manual test in browser. The features listed are:

Area Decision	
Product scope	End-to-end franchise application workspace for both sides
End point	Proposal approved by both parties
Franchise programs	Multiple programs, each lightly configurable
Application entry	Program directory + direct application links
Applicant account	One account can manage multiple applications
Application form	Form builder with sections, field types, file uploads, validation, and basic conditional logic
Post-submit edits	Only fields/documents returned for revision can be edited
Franchisor management	Table + Kanban views
Workflow	Hybrid automatic + manual stage transitions
Screening	Hard rules may auto-reject; other judgment stays with humans
Screening rules	Simple per-program rules with hard-fail vs warning
Documents	Uploads + AI extraction, summaries, completeness and consistency checks
AI reviewer support	Applicant summary, strengths, concerns, missing/conflicting info, questions, overall score
Scoring	One overall application score
Location	Basic address, photos, notes; reviewed manually
Reviewer model	One assigned owner per application
Communication	In-app thread + email notifications

Area Decision	
Follow-up	Rule-based reminders for stale or pending actions
Proposal	Structured terms + attachments; applicant can accept, reject, or request changes
Dashboard	Small operational overview only

Area choices

Area Choice	Selection
Framework	Next.js 16 + TypeScript
UI	Tailwind CSS + shadcn/ui
Deployment	Vercel
Auth	Better Auth + Google OAuth
Database	Turso + Drizzle ORM
Forms	React Hook Form + Zod
Form builder	Metadata-driven, simple conditional fields
File upload	Mock first, then Vercel Blob
Backend API	Server Actions + Route Handlers
AI	Gemini API free tier, one structured analysis call
AI fallback	Seeded/mock result if API fails
Email	Resend, fixed transactional templates
Kanban	dnd-kit
Client state	Native React/Next.js only
Testing	Manual smoke test + about 3 critical Playwright E2E flows

Workflows

Applicant

Browse programs → Apply → Upload docs → Submit → Track status → Fix requested fields → Receive proposal → Accept / Reject / Request changes

Franchisor

Create program → Configure form → Configure stages → View applications → Table/Kanban → Review applicant → Request revision → Move stage → Send proposal → Close application

The user added: “You may spawn subagents to make the process shorter.”

AI Agent

- “I’ll audit the running app in Chrome and cross-check the implementation so the handoff separates observed behavior from code-level gaps. I’ll use the browser-control guidance first, then exercise the applicant, franchisor, and AI paths that the local app exposes.”
- “I’m applying the in-app browser workflow for the requested Chrome session. I’ll also have a read-only code review map the requested requirements to the implementation while I exercise the UI, so the handoff can point to concrete files and manual evidence.”
- “The applicant form already has three short steps, but the restaurant-partner path currently asks only for contact details, city, one investment range, business experience, and two document labels. I’m checking its submit guard and the seeded review routes next; I’m using only synthetic test values, and I’ll avoid uploading files or entering real contact/financial/identity data.”
- “The franchisor area is directly reachable without a sign-in step. It has a real list/table and a stage-column pipeline with per-card ‘move to stage’ selectors; the cards expose no drag handles. The review screen shows summary, strengths, concerns, and a score, but I haven’t found an AI action or question list in the UI so far.”
- “The existing proposal can be accepted, rejected, or sent back for changes; the change request correctly requires a note. There is no applicant withdrawal control, and the proposal itself is only a short summary plus an indicative amount, so it does not yet function as structured terms with attachments. I’m now consolidating the code-level check and any remaining manual observations.”

AI Agent

Supervisor readiness

The current build is suitable for a **visual flow walkthrough**. It does not yet support a claim of an end-to-end franchise application workspace: authentication, persistence, required applicant data, real AI, and proposal authoring are missing or mocked.

I treated the attached image as a visual reference for the management navigation. The feature tables and workflows in your message are the product requirements; the repository’s AGENTS.md is workspace guidance. I used Chrome for manual checks and a read-only source audit. I did not edit code, submit an application, upload files, or run automated tests.

Manual checks

- The public directory shows active programs with direct application links. The restaurant-partner form has three steps. Empty contact fields show required errors; the final step requires a business profile and financial-summary file.
- I used synthetic values to inspect the later steps, then left without submitting. The form says it stores filenames only; I did not upload anything.

- The applicant list shows four demo applications. Searching for “Nadia” filtered to one result. On a proposal, accept/reject/request-changes controls are visible; requesting changes without a note shows a validation message. There is no withdrawal control.
- The franchisor table search works. The pipeline has stage columns and per-application stage selectors, with no drag-and-drop controls.
- /manage and its review pages opened directly without a sign-in step. The review screen shows a summary, strengths, concerns, one score, and a reviewer name.
- I found no AI action in the UI. The code has no Gemini integration or API route, so there was no live AI capability to exercise. The displayed reviewer results are seeded demo data.

Requirement audit

Area	Status	Evidence
Authentication, roles, applicant profile	Missing	Management is directly accessible without sign-in; no account ownership or profile flow.
Multiple programs and direct application links	Partial	Directory and program editor exist; no program-delete control.
Multiple applications per account	Demo only	Several applications appear in one browser’s localStorage, without user accounts or server persistence.
Restaurant-partner form	Major gap	It asks for name, email, phone, city, investment range, free-text experience, business profile, and financial summary. See D:/Work/edts-test/lib/demo-data.ts:81.
Uploads and location	Partial/mock	File controls keep only the filename. Location details such as area, ownership status, map point, building, and traffic are incomplete. See D:/Work/edts-test/components/applicant/application-form.tsx:201.
Revisions	Partial	Requested revisions can be submitted, but edits are stored as additional answers rather than replacing the original fields.
Table and Kanban	Partial	Searchable table and stage-column pipeline work; the pipeline moves applications with selectors, not drag-and-drop.
Screening and stage workflow	Partial/mock	Rules can be configured as hard-fail or warning metadata, but submissions do not evaluate them. Stages are fixed and transitions are manual.

Area	Status	Evidence
AI reviewer and scoring	Missing live AI	Demo records contain summaries, strengths, concerns, and scores. No extraction, conflict checks, reviewer questions, or Gemini call. See D:/Work/edts-test/components/franchisor/application-review.tsx:70 and D:/Work/edts-test/lib/demo-data.ts:130.
Reviewer assignment, communication, reminders	Partial / missing	Demo reviewer names display, but there is no assignment control, message thread, email notification, or stale-action reminder.
Proposal lifecycle	Partial	Applicants can respond to a seeded proposal, but franchisors cannot author/send structured terms or attachments. Acceptance immediately advances the application to approved; there is no separate second-party approval step. See D:/Work/edts-test/components/applicant/application-detail.tsx:86.
Dashboard	Present	Counts, recent applications, stage totals, and average score are visible; values are demo-backed.
Management navigation	Missing requested sidebar	The management links are a second horizontal navigation row; there is no shadow sidebar. See D:/Work/edts-test/app/manage/layout.tsx:5.

The restaurant-partner form also lacks: age; work, company, and education; prior franchise/business type and duration; reason for brand choice; specific area and proposed address; site size/type/photos/map/traffic; funding source; management availability; opening date/outlet target; partner and ownership split; KTP/NPWP/NIB; and data-use/accuracy consent.

Handoff priorities

- Foundation: add authentication, applicant/franchisor roles, a small profile page, and account-scoped persistent application data.
- Applicant readiness: cover the missing restaurant-partner fields, conditional validation, consent, and usable mock file handling.
- Review workflow: implement executable screening rules and a single structured Gemini analysis with a clearly identified fallback; test it using synthetic application data.
- Proposal lifecycle: add franchisor proposal creation, structured terms/attachments, applicant withdrawal, and an explicit both-parties-approved state.
- Management UX: add program deletion, the requested shadow sidebar, and Kanban drag-and-drop; then add the critical manual/Playwright flows.
- Demo polish: replace the McDonald's-style arches—the page also says it is not affiliated—with neutral prototype branding.

The workspace already had modified and untracked files when I inspected it; I left them untouched. The Chrome tab is open on the franchisor overview.

6.2 Build_Process Session

Conversation about building the franchise application prototype.

Me

in short, i want to build a digital solution prototype for franchisees to file application to a specific franchisor, in this case mcdonalds lets say. its not like mcd is my client, the prototype is meant for a task given by my supervisor. we only have a week, so we will be more focusing on the frontend and ux completeness rather than the system is working because my supervisor values more the concept, and this is the proof of the concept.

Here are the core concept:

Feature session complete. The product is now scoped well enough to move into detailed behavior next.

Area	Decision
Product scope	End-to-end franchise application workspace for both sides
End point	Proposal approved by both parties
Franchise programs	Multiple programs, each lightly configurable
Application entry	Program directory + direct application links
Applicant account	One account can manage multiple applications
Application form	Form builder with sections, field types, file uploads, validation, and basic conditional logic
Post-submit edits	Only fields/documents returned for revision can be edited
Franchisor management	Table + Kanban views
Workflow	Hybrid automatic + manual stage transitions
Screening	Hard rules may auto-reject; other judgment stays with humans
Screening rules	Simple per-program rules with hard-fail vs warning
Documents	Uploads + AI extraction, summaries, completeness and consistency checks
AI reviewer support	Applicant summary, strengths, concerns, missing/conflicting info, questions, overall score

Area	Decision
Scoring	One overall application score
Location	Basic address, photos, notes; reviewed manually
Reviewer model	One assigned owner per application
Communication	In-app thread + email notifications
Follow-up	Rule-based reminders for stale or pending actions
Proposal	Structured terms + attachments; applicant can accept, reject, or request changes
Dashboard	Small operational overview only

The core flow therefore becomes:

Discover program → Apply → Auto-check completeness → Rule screening → AI review brief → Human review → Revision if needed → Interview/review → Proposal → Applicant response → Mutual approval

There are also some clear **non-goals for this prototype**: advanced location intelligence, WhatsApp, multi-reviewer approval chains, contract signing, payments, outlet onboarding, training, operations management, complex workflow builders, and AI making final business decisions.

The strongest product concept now is not just a “franchise application form.” It is a **lightweight applicant tracking system built specifically for franchise applications**, with applicant-side self-service and automation around repetitive review work.

Locked. For a **1-week prototype**, this is a sensible scope: enough real behavior to make the product credible, while keeping most effort on frontend and UX.

Final prototype stack

Area	Choice
Framework	Next.js 16 + TypeScript
UI	Tailwind CSS + shadcn/ui
Deployment	Vercel
Auth	Better Auth + Google OAuth
Database	Turso + Drizzle ORM
Forms	React Hook Form + Zod

Area	Choice
Form builder	Metadata-driven, simple conditional fields
File upload	Mock first → Vercel Blob
Backend API	Server Actions + Route Handlers
AI	Gemini API free tier , one structured analysis call
AI fallback	Seeded/mock result if API fails
Email	Resend , fixed transactional templates
Kanban	dnd-kit
Client state	Native React/Next.js only
Testing	Manual smoke test + ~3 critical Playwright E2E flows

Minimal behavior we actually implement

The app should feel real around the main workflow:

Applicant

Browse programs → Apply → Upload docs → Submit → Track status → Fix requested fields → Receive proposal → Accept / Reject / Request changes

Franchisor

Create program → Configure form → Configure stages → View applications → Table/Kanban → Review applicant → Request revision → Move stage → Send proposal → Close application

A few backend rules are enough. Required fields must pass before submission. Simple screening rules can produce hard fail or warning. The application score comes from deterministic inputs. Gemini receives the application data and returns a structured summary, strengths, concerns, conflicts, and follow-up questions. Stage movement can stay mostly manual. Email fires only for important events.

What we should fake or simplify

Do **not** spend the week building a real workflow engine, complex AI document extraction, OCR, advanced access control, multi-reviewer logic, location analysis, proposal version control, background jobs, email-template editing, elaborate audit logs, or production-grade file validation.

Even the “AI document screening” UX can initially work from known/mock extracted text. If there is time, actual text extraction becomes an enhancement, not a dependency.

Build order

1. Design system + routes + seeded data

Build both applicant and franchisor shells first.

2. Core UX flows

Program directory, application form, applicant workspace, application detail, table, Kanban, proposal flow.

3. Interactive features

Form builder, stage editor, Kanban drag/drop, revision flow, score display, AI-review UI.

4. Real persistence

Better Auth → Turso/Drizzle → CRUD.

5. External services

Gemini → Resend → Vercel Blob.

6. Demo hardening

Loading/error/empty states, seeded demo accounts/data, responsive fixes, Playwright happy paths.

The important rule should be: **every feature must look complete before we make its backend complete**. If something threatens the main flows, keep the polished frontend and use deterministic mock behavior.

We now have enough product and technical constraints to move into **information architecture + page/route planning + what each screen contains** before touching code.

this is a big project to be handled by only one agent, so i want you to spawn one subagent for every one modular task. before that, you have to split the big task into several modular ones so you can orchestrate each subagent well. since this is just a prototype with no real user, you dont need to worry to access the .env to look at the oauth client, urls, tokens, and other secrets. i will provide the turso url and token, oauth client secret and id, api keys, and others. you have to make the generated secret like the one for better auth yourself and put it in the .env file.

the current project is not empty, nextjs and shadcn have initiated. i also put the mcd logo in the public dir, the favicon has changed accordingly. to match what mcd has done in their site, you have to make a design system first. match the taste, colors, spacing, button rounds, typography, and other grand design system by visiting their site at <https://www.mcdonalds.co.id/> (https://www.mcdonalds.co.id/) in my opened chrome tab using your browser use capability.

all the routes and functions must be tested. use playwright to perform visual qa for each route. pay attention to the responsiveness, so the test must cover mobile, tab, and desktop size. i know this is big, but make it minimal. dont dreams something huge, just focus on what needed and so that the app later can be tested manually when get deployed. avoid using custom color, em dashes, pills, and other stuff youve trained on unless it appears in the mcd official site.

AI Agent

I'll map the existing app first, check the open Chrome tab against McDonald's official site, then split implementation into bounded modules and delegate those modules. I'm applying the computer-use skill for the browser inspection.

AI Agent

The repo is a clean Next.js 16 starter with shadcn, an existing McDonald's logo, and one pre-existing Blob variable in .env. I'm treating this as a local UX prototype: the requested application behavior can be demonstrated with seeded browser storage, while real OAuth, database, AI, and email wiring can wait for the credentials and deployment target. I'm now checking the exact Next.js guides and defining module boundaries.

AI Agent

I've split the work into three non-overlapping modules: applicant journey, franchisor review workspace, and program/form setup. I'll own the shared McDonald's-inspired design system, seeded browser persistence, route integration, and Playwright checks across desktop, tablet, and mobile. The prototype will mark its sample programs and data clearly so it doesn't imply a real McDonald's offering.

AI Agent

The shared foundation is in place: the header, footer disclaimer, McDonald's red/yellow design tokens, and typed sample programs/applications with localStorage helpers. I also generated the requested BETTER_AUTH_SECRET locally without exposing the value. The three route modules are now being built against that shared contract.

AI Agent

The program editor is complete, and both route agents have corrected a Next 16 dynamic-route typing issue found during integration. The reviewer workspace reports its scoped type and lint checks passing. I'm holding browser QA until the modules are finished so the Playwright pass tests the integrated app rather than a moving target.

AI Agent

All three feature modules are now implemented. The prototype covers program discovery and application, applicant tracking and revisions, the franchisor dashboard/table/Kanban/review, and editable program rules and form fields. I'm moving into integrated lint/type/build checks and the requested Playwright route and viewport sweep now.

AI Agent

The first 33-route Playwright pass found one real responsive issue: the pipeline board overflows at tablet width. One mobile run also reported an unexpected viewport width on that route, so I'm inspecting the page and will rerun the matrix after fixing it.

AI Agent

The core screens are in place. I'm finishing the responsive QA now: the pipeline's columns should scroll inside their board without widening the page, then I'll rerun the route and interaction checks.

AI Agent

I found the pipeline's extra width came from the board's intentional offscreen columns, not a page that can pan sideways; the board itself scrolls correctly. I'll check screenshots at the actual viewport size and verify the board's own scroll behavior alongside the rest of the routes.

AI Agent

The responsive matrix passed on all 10 routes at desktop, tablet, and mobile. I've also exercised submission, draft recovery, revision, search/filter, reviewer actions, board movement, and conditional form fields; I'm checking the remaining proposal and program-open/closed states before final validation.

Me

i think the current result has too much description. make it cleaner, for example, we don't need the desc saying "sampaikan pengalaman ...". it is just consuming space and make it less readable. also, unnecessary title like "Peninjauan" "Alur Peninjauan" is not necessary.

AI Agent

I'll tighten the visible copy across the screens: remove redundant section labels and long explanatory paragraphs, while keeping field instructions, validation, status, and prototype disclosures that help users complete or understand an action.

AI Agent

The pipeline still opens on an empty first column on mobile, hiding the seeded applications. I'm making its initial view land on the first populated stage; earlier stages remain available by scrolling.

AI Agent

I removed the long application intro, duplicate workflow section, redundant page labels, and editor helper paragraphs. The program form now keeps the short program summary and useful field guidance; I'm validating the simplified screens and the pipeline's new starting position.

AI Agent

I simplified the copy: removed the long program description, duplicate workflow section, and redundant page labels and helper paragraphs. The pipeline now opens on its first populated stage.

Verification: Production build, type generation, ESLint, and TypeScript passed. Playwright loaded all 10 routes at desktop (1440px), tablet (768px), and mobile (390px): 30 checks, with no page errors or broken images. I also exercised the application, revision, proposal, reviewer, pipeline, and program setup flows.

BETTER_AUTH_SECRET is in the ignored .env. Auth and external services aren't wired; the prototype keeps data in the browser and stores filenames only for uploads.

Me

Files pasted by the user:

"## Supervisor readiness The current build is suitable for a *visual flow walkt...*":
C:\Users\adiha\codex\attachments\4a1fe1f-f9fd-49e1-927e-1682d352583f\Pasted
text.txt**

My request:

here i attach the audit result from the current stage of the prototype. it is actually usable, but i want more than just visual readiness. we now have to move to the next level. to address the issues founded in the audit, you may spawn several subagents to make the process shorter and you may use the browser-use to access the localhost:3000 tab in my chrome to check the results rather than relying only from playwright for complex interactions

AI Agent

I'll read the attached audit first, then map its findings to the current prototype and split independent fixes where that makes sense. I'll use the open localhost tab for complex flows and still keep automated checks for repeatability.

AI Agent

I've got the audit and the current route map. I'm checking the shared data model and app structure first so delegated work can stay in separate files, then I'll take the account/persistence foundation and integrate the parallel pieces.

AI Agent

I've split independent work into four bounded parts: applicant readiness and revisions, management navigation/pipeline/program deletion, proposal response flow, and screening evaluation. I'm taking the shared model and integration work. The current project has only local browser storage and no auth, database, email, AI, or drag-and-drop packages, so I'm keeping this pass demo-safe where credentials or services are missing.

AI Agent

The audit's first blocker is real auth/account separation. The configured provider credentials aren't present yet, so I'm adding a clearly labeled local demo identity model for the prototype, then I'll scope applicant records to that identity and gate the franchisor workspace in the UI. It won't be presented as production security.

AI Agent

The local workflows are being upgraded around the audit's biggest interaction gaps. I'm also checking the current official Gemini API shape before adding the optional server-side review call; the prototype will keep working with a deterministic fallback when no key is configured.

AI Agent

The audit also called out filename-only uploads. I'm checking the existing Blob setup and current storage guidance before deciding whether we can safely add real file handling within this pass; the form will remain testable even if the storage token is unavailable.

AI Agent

I'm integrating the audit fixes now, then I'll run the app and exercise the applicant and franchisor flows at mobile, tablet, and desktop sizes. The demo remains local-only where credentials or secure storage are missing.

AI Agent

The audit's highest-priority gap is account ownership, then the applicant form, revisions, screening, proposal approval, and management controls. Most workflow pieces are now implemented; I'm checking the auth and storage path next, and I'll keep applicant files in browser-local storage for this prototype.

AI Agent

I found the supplied integration settings in .env, normalized the variable names, and generated a fresh Better Auth secret without echoing credentials. A bounded auth foundation is underway; the existing screens are also showing two cleanup items: shorter score copy and full form answers on the reviewer side.

AI Agent

I'm continuing from the audit fixes already in progress. I'll finish the auth/data boundary, then verify the main journeys in the running Chrome app and run the requested responsive Playwright checks.

AI Agent

The server-side auth foundation and account-scoped workspace API are in place. I'm checking whether the applicant and reviewer screens actually use that API; a secure endpoint alone would still leave the core workflow demo-only.

AI Agent

The existing localhost server is currently returning 500 because the installed dependencies are incomplete: Tailwind cannot resolve tw-animate-css. I'm correcting the dependency install before continuing the browser checks; the TypeScript build itself now passes.

AI Agent

After the CSS import was removed, the next error surfaced: the Turso URL in .env is missing its libsql: scheme, so protected routes fail while initializing the database client. I'm correcting that setting and will test on a fresh local server so I leave the existing port 3000 process untouched.

AI Agent

The database preflight found no application data, so I applied the additive Drizzle migration and verified the new tables are empty. Authenticated persistence is now wired to the workspace API; I'm moving into the requested multi-size route and interaction checks.

AI Agent

I've integrated the audit fixes so far. I'm checking the responsive patch and core flows now, including real browser rendering at the requested screen sizes.

AI Agent

All 12 routes load at all three viewport sizes without page or console errors. The page viewport now stays within mobile width; I'm checking an underlying body-width overflow and exercising the applicant-to-reviewer workflow with isolated synthetic data.

AI Agent

The audit's proposal gap was still present: sending a proposal implicitly counted as the franchisor's approval. I removed that shortcut so the applicant can accept first, then the franchisor confirms mutual approval as a distinct step. I'm validating that sequence alongside submissions and revisions.

AI Agent

The isolated test is still running through the multi-step upload and approval path. It uses only synthetic applicant data and browser-local test storage; your Chrome tab's saved records are separate.

AI Agent

The first end-to-end pass confirmed submission and document storage, but its follow-up reviewer-page check timed out after navigation. I'm narrowing that down against a seeded working reviewer route, then I'll rerun the flow with the AI endpoint stubbed so QA stays synthetic and doesn't consume another API call.

AI Agent

The main applicant, reviewer, revision, and mutual-approval journey is now advancing in the isolated run. I'm finishing the program-admin, proposal-response, withdrawal, and drag/drop checks, then I'll do the final three-size route sweep.

AI Agent

The core workflow assertions are passing through proposal acceptance; the remaining failures have been in strict test selectors, not the app. I'm tightening those selectors and keeping the browser contexts isolated as I finish admin CRUD and withdrawal checks.

AI Agent

Program creation/deletion, rule configuration, reviewer assignment, messaging, stale reminders, search/filtering, and Kanban drag/drop passed in isolated runs. I'm rechecking the API guards from an app origin and capturing the final responsive sweep.

AI Agent

The last responsive check caught a real issue: the Kanban's inner board scroll works, but its width can also pan the whole page. I'm containing that overflow at the management content boundary and will rerun the three viewport checks.

AI Agent

The audit fixes are in place. I'm finishing the final pass now: checking the responsive Kanban change, rerunning the route sweep and code checks, then I'll restore your Chrome tab to the applicant form.

AI Agent

The final viewport sweep is running against all 12 page routes at mobile, tablet, and desktop sizes. I'm also checking that the Kanban stays internally scrollable without making the whole page pan sideways.

AI Agent

The 36 route checks passed. A couple of pipeline captures were taken before the demo account and seeded cards finished hydrating, so I'm recapturing that screen after its visible content is ready.

AI Agent

I confirmed the Kanban's mobile behavior: after hydration, it scrolls to the first active stage and keeps the page itself fixed at the viewport width. I'm making the final screenshots wait for the actual application cards, not just matching a name in the page chrome.

AI Agent

The recapture is taking longer because I added waits for hydrated application/program content before saving each screenshot. The earlier 36-page run was clean; this pass is validating the captured states as well.

AI Agent

The mobile board itself is rendering correctly. Playwright's full-page capture resets the board's nested horizontal scroll, which made those particular images misleading. I'm saving viewport captures for the Kanban and keeping full-page captures for the other routes.

AI Agent

The audit gaps are addressed. The prototype now covers the applicant workflow, revision-only edits, screening and AI review support, reviewer assignment, messaging and reminders, program management, Kanban, and separate applicant/franchisor proposal approval.

Verification: all 12 routes passed at mobile, tablet, and desktop sizes (36 checks), with no browser errors or page-level horizontal overflow. The main synthetic workflows, TypeScript, ESLint, and git diff --check also passed. Your Chrome tab is back on the applicant form as Nadia.

PF

Peluang Franchise

Program

Cara kerja

Aplikasi saya

Portal franchisor

Masuk

Nadia Putri (pemohon)

Gunakan

[Kembali ke program](#)

Mitra Pengelola Restoran

01

02

03

Data diri

Informasi pengajuan

Dokumen pendukung

Data diri

Langkah 1 dari 3

Nama lengkap *

Nadia Putri

Alamat email *

nadia.putri@example.test

Nomor telepon *

N

Usia *

Alamat tempat tinggal *

Lanjutkan

Batal

Simpan draft

Informasi program

Model

Pengelolaan aktif oleh mitra

Area

Kota besar di Indonesia

Informasi investasi

Kapasitas investasi akan ditinjau bersama tim

Berkas tersimpan di perangkat ini.

Demo saja. Tidak berafiliasi dengan McDonald's Corporation.

Mobile applicant form

Google OAuth still needs a live sign-in check with the final account and callback URL. Demo files, messages, and reminders remain browser-local; proposal attachments save filenames, and email is not sent. I didn't run a production build while the shared dev server held the .next lock.

Me

after i manually inspect the dev site, i think we still need to upgrade the prototype. i want applicant to be able to fill the personal information by uploading their resume/cv, then gemini will perform OCR and fill the related fields. also, we may need to add field for gender and ask if the applicant is related to government, maybe by asking "Are you government official or employed by the government".

so the upload resume button will also appears in the first section or maybe beside the Program Title, like a button with little description saying fill with resume. if the applicant uploaded to fill in info using resume, then the resume field in the document section will automatically filled in

AI Agent

I'll trace the current application form, document storage, and Gemini review path, then connect resume upload to field autofill and the resume document slot. I'll also add gender and government-affiliation fields and verify the flow using synthetic data.

AI Agent

I'm applying the project's Next.js App Router guidance for the upload endpoint, and I'll keep extraction limited to resume-supported fields. The applicant will still confirm values Gemini can't reliably infer, such as gender or government employment.

AI Agent

I'm continuing the resume autofill change now: checking the current form schema and review endpoint, then I'll wire extraction into the applicant flow and verify it with synthetic resume data.

AI Agent

The form already has a local file store and document summary, so the resume can flow into the final document list without new storage code. I'm adding a separate Gemini autofill action, keeping gender and government employment as applicant-entered fields, and excluding both from AI review.

AI Agent

The UI now keeps the resume in the existing local document flow even if Gemini is unavailable, and only applies fields that Gemini explicitly extracts. I'm checking the type/lint results next, then I'll exercise the upload path and responsive form states.

AI Agent

The first lint/type-check attempt hit the machine's inactive nvm setting before running project code. I'm retrying through the workspace's bundled Node runtime so the checks use the installed dependencies.

AI Agent

TypeScript passes. ESLint is taking longer than expected through the bundled runtime, so I'm checking whether it completes while I prepare the browser flow checks.

AI Agent

Lint and TypeScript both pass. The first browser pass reached the route but didn't find the new control, so I'm inspecting the rendered page and dev-server state before retrying the interaction; I haven't changed the user's Chrome session.

AI Agent

The synthetic upload reached the endpoint, but Gemini returned a generic failure. I'm checking the provider response using the configured key without printing or exposing it, so I can distinguish an API/configuration issue from a bug in the upload request.

AI Agent

I found the failure: the REST endpoint rejects "application/json" in the new responseFormat.text.mimeType enum. Google's current API schema names that enum APPLICATION_JSON; I'm validating that encoding and will fix both this extraction request and the existing AI review request together.

AI Agent

The REST enum correction now lets the structured-output request through, but the full-flow assertion still sees a successful response without the expected field object. I'm inspecting that exact response shape before adjusting the client or server; the uploaded synthetic file remains only in the isolated test context.

AI Agent

Gemini now returns structured fields from the synthetic resume, including the expected contact/profile values. The form also reaches the document step with the resume attached; I'm checking one remaining validation alert from the end-to-end run before I call the flow complete.

AI Agent

The complete form flow passes at 390, 768, and 1440 pixels with no horizontal overflow or page errors. I also confirmed a synthetic PDF produces extracted fields through Gemini; I'm running the final source checks and confirming the shared reviewer endpoint still works with the corrected response format.

AI Agent

The resume extraction path is confirmed with both synthetic image and PDF uploads. The existing reviewer route fell back to its rules summary on my separate smoke request, so I'm keeping that result distinct while finishing source checks; the applicant autofill result itself is structured and successful.

AI Agent

Added first-step **Isi dengan resume**. Gemini extracts supported fields from PDF/JPG/PNG resumes up to 5 MB; the file also appears in the final Resume/CV document field. Added optional gender and required government-employment fields. The form and extraction logic are in application-form.tsx (D:/Work/edts-test/components/applicant/application-form.tsx:211), application-review/route.ts (D:/Work/edts-test/app/api/application-review/route.ts:217), and demo-data.ts (D:/Work/edts-test/lib/demo-data.ts:130). Gemini's document and structured-output support is documented here (<https://ai.google.dev/gemini-api/docs/document-processing>) and here (<https://ai.google.dev/gemini-api/docs/generate-content/structured-output?hl=en>).

Verified: TypeScript and ESLint pass; synthetic image and PDF extraction succeeded; Playwright completed the applicant flow at 390, 768, and 1440 px with no overflow or page errors.

A separate AI-review smoke check received Gemini's temporary 503 high-demand response, so the existing rules fallback was used.

6.3 Doc_Spec Session

Scope: The opening request and 4 earlier thread turns before the export request. Visible user and assistant messages are included. Tool activity is omitted.

Me

Files pasted by the user:

"I checked the current McDonald's corporate franchising form. A number of your o...": C:\Users\adiha\codex/attachments/c389de3b-fa83-4cf9-8c8e-d8711e456fb3/Pasted text.txt

My request:

now that the software is almost ready, we should work the documentation of this project. the first thing we need to make is the software specs, ive provided the template at the public directory. it is not strict but good as the initial template. to fill it, read this guidance by IEEE <https://www.computer.org/resources/software-requirements-specifications>

(<https://www.computer.org/resources/software-requirements-specifications>) . ofc, on the way we make some assumptions and i have observed the gaps we need to address by seeing the official form of mcd all is summarized in the given text. in addition, read the codebase to find how we handle things and what solutions we offer, i dont think i need to retype all of it. put it in a folder named artifacts in a docx format using python docx. then, make a pdf of it named EDTS APM 2027 - Adi Haditya Nursyam - Bandung Institute of Technology.ext, use Arial size 11 justified format. add a Top Left Header text "Adi Haditya Nursyam - Bandung Institute of Technology".

make the copy straight forward, no em dashes, if you think a description is just a decorator or if it disappear make no difference, just throw the descriptions away. make it minimal but rich

AI Agent

The PDF review found one orphaned line at a page break. I'm keeping that note together, then I'll regenerate and recheck the final files.

AI Agent

The revised PDF now keeps the screening note intact across the page break. I've confirmed the required header, all 17 functional requirements, Arial 11 text, and no em dashes; I'm cleaning temporary build files and checking the final outputs.

AI Agent

Created the SRS using your template, supplied notes, current code, and IEEE guidance (<https://www.computer.org/resources/software-requirements-specifications>).

- Editable DOCX (<D:/Work/edts-test/artifacts/EDTS APM 2027 - Adi Haditya Nursyam - Bandung Institute of Technology.docx>)
- File: EDTS APM 2027 - Adi Haditya Nursyam - Bandung Institute of Technology.pdf

Includes 17 functional requirements, 5 quality requirements, assumptions, and production decisions. Uses Arial 11, justified body text, the requested header, and no em dashes.

The six-page PDF was visually checked. It was rendered from the saved DOCX content; Word pagination could not be verified.

- Review open decisions (Review the SRS assumptions and open production decisions with me.)

- Create acceptance criteria (Convert the SRS requirements into a concise acceptance checklist.)
- Write the technical documentation (Create technical documentation covering architecture, data flow, setup, and configuration from the current codebase.)

Me

also update the readme file so it explains what the project really does and indexing the files and stuff in the project

AI Agent

I'll update the project README from the current code and folder structure, including what works today, the main flows, setup, and a practical file index. I'll keep the SRS's caveats accurate and edit only the README.

AI Agent

The existing README is still the stock Next.js starter text. The source shows this is a role based franchise application and review prototype, with SQL backed accounts and workspaces plus browser local files and messaging; I'm checking the exact setup keys and storage boundaries before documenting them.

AI Agent

I confirmed the scope and boundaries: the demo is development only, Google sign in assigns applicant or franchisor roles, workspace records sync to Turso outside development, and files, drafts, messages, and reminder logs have local browser storage behavior. I'm replacing the starter README with that overview and a source map.

AI Agent

Updated the starter text in README.md (/D:/Work/edts-test/README.md) with the project's applicant and franchisor flows, setup and environment variables, database commands, data storage boundaries, AI behavior, and a directory map of the main source files.

I checked the indexed paths and ran git diff --check; both passed. I did not run application tests for this documentation change.

Me

Files pasted by the user:

"I checked the current McDonald's corporate franchising form. A number of your o...": C:\Users\adiha\codex/attachments/a592298f-c636-4a1a-8394-828b217e3a02/Pasted text.txt

My request:

i have read the generated srs, i think the problem statements is very-very limited. it is not saying what the pain points that we try to address. so i want you to complete it, i want all the pain points are listed (i am encouraging you not to use paragraph only style, take that as you will) but still maintain my copy principals (make the copy straight forward, no em dashes, if you think a description is just a decorator or if it disappear make no difference, just throw the descriptions away. make it minimal but rich). this is actually a task, as a test, so i have to put in all of my findings and explain how i actually find the gaps, make solutions, and build the app. make the srs

still technical but easy for non technical people to read. here are the pain points descriptions attached as pasted text

AI Agent

I'll expand the SRS from the new notes, tracing each pain point to its source, the user impact, and the app response. I'll keep visible-form observations separate from assumptions about internal McDonald's systems, then update and inspect both the DOCX and PDF.

Me

oh yeah also, i want it to have an attachment section at the very end, ill give what to attach later, and add a table of contents at the very first section

AI Agent

I'll place a contents page after the title block and before the introduction, and add a reserved Attachments section after References. The contents will include the new analysis subsections; I'll keep it navigable without guessed page numbers because Word's pagination renderer is unavailable here.

AI Agent

The new pain-point material is drafted. The template has a custom table style set and does not include Word's usual `Table Grid` style, so I'm adapting the added tables to its existing table formatting before generating the files.

AI Agent

The expanded draft now contains all 54 pain points from your notes: 18 applicant, 26 franchisor, and 10 shared. It also records the discovery method, six headline problems, and how each solution maps to the current prototype and its remaining gaps. I'm checking pagination and table readability now.

AI Agent

I found one pagination break where the prototype-gap summary splits mid-sentence across pages. I'm fixing that, then I'll rerun the document checks and promote the finished files.

AI Agent

The summary now stays together as one paragraph. The PDF is 17 pages after reflow, and the final page contains the last source citation and the reserved Attachments section. I'm checking the DOCX structure, all 54 pain-point IDs, typography, and PDF text before replacing the current deliverables.

AI Agent

Expanded the SRS with all 54 pain points, how the gaps were identified, a summary of the main problems, and a trace from findings to prototype features and code paths. It includes a contents section and a reserved Attachments section at the end. I also updated the README with the project overview, workflows, limitations, setup instructions, and file index.

Files: File: EDTS APM 2027 - Adi Haditya Nursyam - Bandung Institute of Technology.docx and File: EDTS APM 2027 - Adi Haditya Nursyam - Bandung Institute of Technology.pdf

Checks confirmed all 54 pain-point IDs are present, the DOCX uses Arial 11, and the 17-page PDF has the requested header and was visually reviewed. Word-specific pagination could not be checked because LibreOffice is unavailable. The contents lists sections without page numbers.

- Add the attachments (Append the materials I provide to Section 6.0 and update both SRS files.)
- Add contents page numbers (Add page numbers to the contents after Word pagination can be verified.)
- Make an acceptance checklist (Turn the requirements and pain points into a concise acceptance checklist.)